

AFRILANG Stdlib

std/c/mlgradient

Français. Bibliothèque AFRILANG 'std/c/mlgradient' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : gradSum, gradMean, gradMin, gradMax, gradVar, gradStd, gradNorm.

```
'import "std/c/mlgradient"' · 'use mlgradient'
```

```
- 'gradSum(v list number) → number' - 'gradMean(v list number) → number' - 'gradMin(v list number) → number' - 'gradMax(v list number) → number' - 'gradVar(v list number) → number' - 'gradStd(v list number) → number' - 'gradNorm(v list number) → list number'
```

English.

AFRILANG library 'std/c/mlgradient' (tier complex, category complex). Exports 7 function(s): gradSum, gradMean, gradMin, gradMax, gradVar, gradStd, gradNorm.

```
'import "std/c/mlgradient"' · 'use mlgradient'
```

```
- 'gradSum(v list number) → number' - 'gradMean(v list number) → number' - 'gradMin(v list number) → number' - 'gradMax(v list number) → number' - 'gradVar(v list number) → number' - 'gradStd(v list number) → number' - 'gradNorm(v list number) → list number'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/mlgradient' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : gradSum, gradMean, gradMin, gradMax, gradVar, gradStd, gradNorm.

```
'import "std/c/mlgradient"' · 'use mlgradient'
```

```
- `gradSum(v list number) → number`
- `gradMean(v list number) → number`
- `gradMin(v list number) → number`
- `gradMax(v list number) → number`
- `gradVar(v list number) → number`
- `gradStd(v list number) → number`
- `gradNorm(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlgradient"
use mlgradient
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- gradSum(v list number) → number•
gradMean(v list number) → number•
gradMin(v list number) → number•
gradMax(v list number) → number•
gradVar(v list number) → number•
gradStd(v list number) → number•
gradNorm(v list number) → list

4. Exemples d'utilisation

```
import "std/c/mlgradient"
use mlgradient
```

```
create result = gradSum(/* args */)
say result
```

```
create result = gradMean(/* args */)
say result
```

```
create result = gradMin(/* args */)
say result
```

```
create result = gradMax(/* args */)
say result
```

```
create result = gradVar(/* args */)
say result
```

```
create result = gradStd(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/mlgradient"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module mlgradient

```
function gradSum(v list number) returns number
end
```

```
function gradMean(v list number) returns number
end
```

```
function gradMin(v list number) returns number
end
```

```
function gradMax(v list number) returns number
end
```

```
function gradVar(v list number) returns number
end
```

```
function gradStd(v list number) returns number
end
```

```
function gradNorm(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/mlgradient' (tier complex, category complex). Exports 7 function(s): gradSum, gradMean, gradMin, gradMax, gradVar, gradStd, gradNorm.

'import "std/c/mlgradient"' · 'use mlgradient'

```
- `gradSum(v list number) → number`
- `gradMean(v list number) → number`
- `gradMin(v list number) → number`
- `gradMax(v list number) → number`
- `gradVar(v list number) → number`
- `gradStd(v list number) → number`
- `gradNorm(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlgradient"
use mlgradient
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- gradSum(v list number) → number•
gradMean(v list number) → number•
gradMin(v list number) → number•
gradMax(v list number) → number•
gradVar(v list number) → number•
gradStd(v list number) → number•
gradNorm(v list number) → list

4. Usage examples

```
import "std/c/mlgradient"
use mlgradient
```

```
create result = gradSum(/* args */)
say result
```

```
create result = gradMean(/* args */)
say result
```

```
create result = gradMin(/* args */)
say result
```

```
create result = gradMax(/* args */)
say result
```

```
create result = gradVar(/* args */)
say result
```

```
create result = gradStd(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/mlgradient"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module mlgradient

```
function gradSum(v list number) returns number
end
```

```
function gradMean(v list number) returns number
end
```

```
function gradMin(v list number) returns number
end
```

```
function gradMax(v list number) returns number
end
```

```
function gradVar(v list number) returns number
end
```

```
function gradStd(v list number) returns number
end
```

```
function gradNorm(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mlgradient"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/mlgradient/>

Source (extrait)

```
module mlgradient

function gradSum(v list number) returns number
end

function gradMean(v list number) returns number
```

```
end

function gradMin(v list number) returns number
end

function gradMax(v list number) returns number
end

function gradVar(v list number) returns number
end

function gradStd(v list number) returns number
end

function gradNorm(v list number) returns list number
end
end
```